TurtleBot3 Motion Server

Design, Setup, and Operation Guide

A browser-based teleoperation and camera-streaming system

`ttb-experiments`

August 2, 2026

Abstract

This document explains, from the ground up, a system that lets you **drive a TurtleBot3 robot from a web browser** while watching its camera in real time. It assumes *no prior knowledge of ROS* (the Robot Operating System) or robotics. It covers the underlying concepts, the software that was written, the network and configuration issues that were solved along the way, and complete step-by-step instructions to start, use, and stop the system.

Contents

1 Introduction

Imagine a small wheeled robot with a camera on top. This project lets you open a web page on your laptop, see through the robot's camera, and press buttons (or use the keyboard) to drive it around — forward, backward, and turning — all wirelessly. We call this **teleoperation**, or "teleop" for short: operating a machine from a distance.

The whole thing is one Python program, `motion_server.py`, that runs on a laptop. It does three jobs at once:

1. Receives the live camera images from the robot and shows them in the browser.
2. Sends your button presses to the robot as movement commands.
3. Serves a web page (the graphical user interface, or **GUI**) that ties it all together.

The equipment. There are two computers involved:

- The **TurtleBot3** (model "Burger") — a small robot whose brain is a Raspberry Pi. It has two motors, a spinning distance sensor (a lidar), and a USB camera.
- Your **laptop** — where the web server runs and where you open the browser.

They talk to each other over Wi-Fi.

2 A Crash Course for Total Beginners

This section explains just enough background to understand everything else. If you already know ROS, skip to Section 3.

2.1 What is ROS 2?

ROS 2 (Robot Operating System 2) is not really an operating system — it is a *toolkit* for building robot software. Its most important idea is that a robot's software is split into many small programs called **nodes**, and these nodes talk to each other by passing **messages**.

2.2 Topics: the robot's "radio channels"

Nodes communicate over named channels called **topics**. Think of a topic as a radio frequency with a name, like `/cmd_vel` or `/image`.

- A node that **publishes** to a topic is a broadcaster.
- A node that **subscribes** to a topic is a listener.

Any number of listeners can tune in. In our system:

- The robot's camera node **publishes** pictures on the topic `/image/compressed`. Our server **subscribes** to it.
- Our server **publishes** movement commands on the topic `/cmd_vel`. The robot's motor node **subscribes** to it.

2.3 Messages: Twist and images

Every message on a topic has a fixed shape (a "type"):

- A movement command is a **Twist** message. It carries a *linear* speed (how fast to go forward/backward, in metres per second) and an *angular* speed (how fast to spin, in radians per second). Sending `linear=0.1, angular=0` means "drive forward gently." Sending all zeros means "stop."

- A camera picture is an **Image** (or **CompressedImage**, which is the same picture squeezed smaller — like a JPEG — so it travels faster over Wi-Fi). Our robot sends the compressed kind.

2.4 How two computers find each other: the Domain ID

ROS 2 nodes discover each other automatically over the network using a system called **DDS**. To keep separate robot projects from interfering, every node belongs to a **ROS_DOMAIN_ID** — a number from 0 to 232. *Two computers can only see each other's topics if they share the same ROS_DOMAIN_ID.* In this project both the robot and the laptop use **domain 203**. (A mismatch here was one of the first bugs we hit — see Section 7.)

3 System Architecture

Figure 1 shows how data flows. The camera images travel *from* the robot *to* the browser; the drive commands travel the other way.

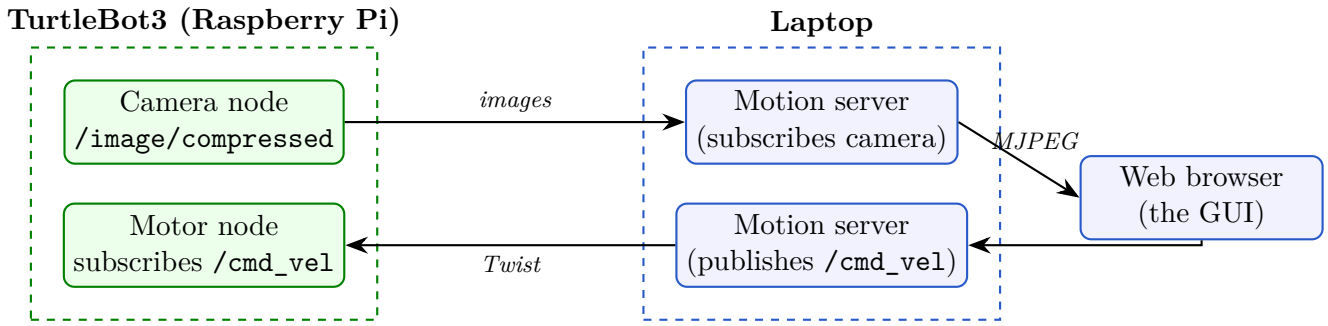

Figure 1: Data flow. Left: the robot. Right: the laptop. Camera images flow right; drive commands flow left. Wi-Fi carries the ROS 2 traffic between them.

The two dashed boxes are the two computers. Inside the laptop, the single `motion_server.py` program plays several roles: it subscribes to the camera, re-serves those frames to the browser as a video stream, and turns your clicks into `/cmd_vel` messages.

4 Hardware and Network Configuration

Table 1 lists every setting the system depends on. All of these live in a single `.env` file so they are easy to change (see Appendix A).

5 ROS 2 Topics and Interfaces Reference

Section 2.4 introduced topics as "radio channels." This section is the complete reference for the channels this robot actually broadcasts.

A point that confuses newcomers: *topics are not defined in configuration files.* There is no file you can open that lists them. A topic springs into existence when a running node asks for it in code — for example, the camera node creates its channel with the single line

```
self.publisher_ = self.create_publisher(CompressedImage, '/image/compressed', 1)
```

(see `robot/.../image_publisher.py`, line 38). So the authoritative list of topics comes from asking the *live* robot, not from reading a file. Table 2 is exactly that: the output of `ros2 topic`

Setting	Value	Meaning
Robot IP	`192.168.68.100`	Address of the robot on the Wi-Fi
Laptop IP	`192.168.68.113`	Address of the laptop
Robot login	`ubuntu`	SSH username on the robot
ROS_DOMAIN_ID	`203`	Shared "channel group" (Sec. 2.4)
Robot model	`burger`	The TurtleBot3 variant
Lidar model	`LDS-01`	The distance sensor type
Camera topic	`/image/compressed`	Where images are published
Command topic	`/cmd_vel`	Where drive commands go
Max linear speed	`0.22` m/s	Burger's top forward speed
Max angular speed	`2.84` rad/s	Burger's top turning speed
Server port	`8000`	Web address is `localhost:8000`

Table 1: System configuration.

`list` on a running system, with each type resolved.

Topic	Message type	Published by
`/cmd_vel`	`geometry_msgs/msg/Twist`	**this app** (laptop → robot)
`/image/compressed`	`sensor_msgs/msg/CompressedImage`	`image_publisher` (custom)
`/odom`	`nav_msgs/msg/Odometry`	`diff_drive_controller`
`/scan`	`sensor_msgs/msg/LaserScan`	`hlds_laser_publisher`
`/imu`	`sensor_msgs/msg/Imu`	`turtlebot3_node`
`/magnetic_field`	`sensor_msgs/msg/MagneticField`	`turtlebot3_node`
`/battery_state`	`sensor_msgs/msg/BatteryState`	`turtlebot3_node`
`/joint_states`	`sensor_msgs/msg/JointState`	`turtlebot3_node`
`/sensor_state`	`turtlebot3_msgs/msg/SensorState`	`turtlebot3_node`
`/tf`, `/tf_static`	`tf2_msgs/msg/TFMessage`	`robot_state_publisher`
`/robot_description`	`std_msgs/msg/String`	`robot_state_publisher`

Table 2: Every topic on the running system (`ROS_DOMAIN_ID=203`).

Of these eleven, the motion server touches exactly **two**: it subscribes to `/image/compressed` and publishes to `/cmd_vel`. Everything else comes free with the standard bringup and is there if you want to extend the project — `/scan` for obstacle avoidance, `/odom` for tracking where the robot has driven, `/battery_state` for a charge warning.

5.1 Topics versus services

A **topic** is one-way broadcast: the sender publishes and never learns who listened. A **service** is a two-way request/response, like a phone call — you ask a question and wait for an answer.

Services *are* defined in files, which is why the distinction matters here. The robot package contains one, `srv/Motion.srv`, which declares a request of two floats and a response of a success flag plus a message. It belongs to the upstream Jetson AI stack and **is not used by this app** — the web GUI drives the robot purely by publishing `Twist` messages on `/cmd_vel`.

5.2 Inspecting topics yourself

```
export ROS_DOMAIN_ID=203
ros2 topic list # every topic currently on the network
ros2 topic type /cmd_vel # what message type it carries
ros2 topic echo /odom # print messages as they arrive
```

```
ros2 topic hz /scan # measure how fast they arrive
ros2 node list # which programs are running
```

If `ros2 topic list` shows only `/parameter_events` and `/rosout`, it is almost always a **stale ROS daemon**, not a network fault — and it will hide topics even on the robot itself. Fix it with `ros2 daemon stop`, or add `--no-daemon` to the command. Note that `ros2 topic hz` does *not* accept `--no-daemon` on Humble: it exits with a usage error that looks deceptively like "no data." Discovery also needs 20–30 seconds to fully settle after launch, so an incomplete list immediately after starting the robot is normal.

6 What Was Built: The Motion Server

The entire application is the file `motion_server.py`. Here is what each part does, in plain language.

6.1 Camera streaming

The server subscribes to the robot's camera topic. Each incoming compressed image is decoded into a picture. A special web address, `/video`, streams these pictures to the browser one after another (a technique called **MJPEG**), which the browser shows as a live video. The program can handle both "raw" and "compressed" image types and detects which one automatically.

6.2 Driving the robot

When you press a direction, the browser calls the address `/cmd` with a requested linear and angular speed. The server clamps those numbers to the robot's safe maximums and remembers them as the "target velocity."

6.3 Safety: the dead-man's switch

A robot that keeps moving after you lose connection is dangerous. To prevent this, the server re-sends the target velocity 20 times per second, but if no fresh command has arrived in the last **0.4 seconds** it automatically sends *stop*. In practice this means: the robot moves only while you are actively holding a button or key; the instant you release — or your browser or Wi-Fi drops — it halts. This is called a **dead-man's switch**.

6.4 The web GUI

Figure 2 shows the interface. On the left is the live camera. On the right is the control panel:

- A 3×3 grid of labelled buttons: *Forward, Backward, Turn Left, Turn Right*, the four diagonals, and a large red **STOP** in the centre.
- A live velocity read-out (v and ω).
- Sliders to set how fast forward driving and turning are.
- A latency read-out (explained in Section 6.5).

You can also drive with the keyboard: **W/A/S/D** or the arrow keys, and **Space** or **X** to stop. Press-and-hold to move; release to stop.

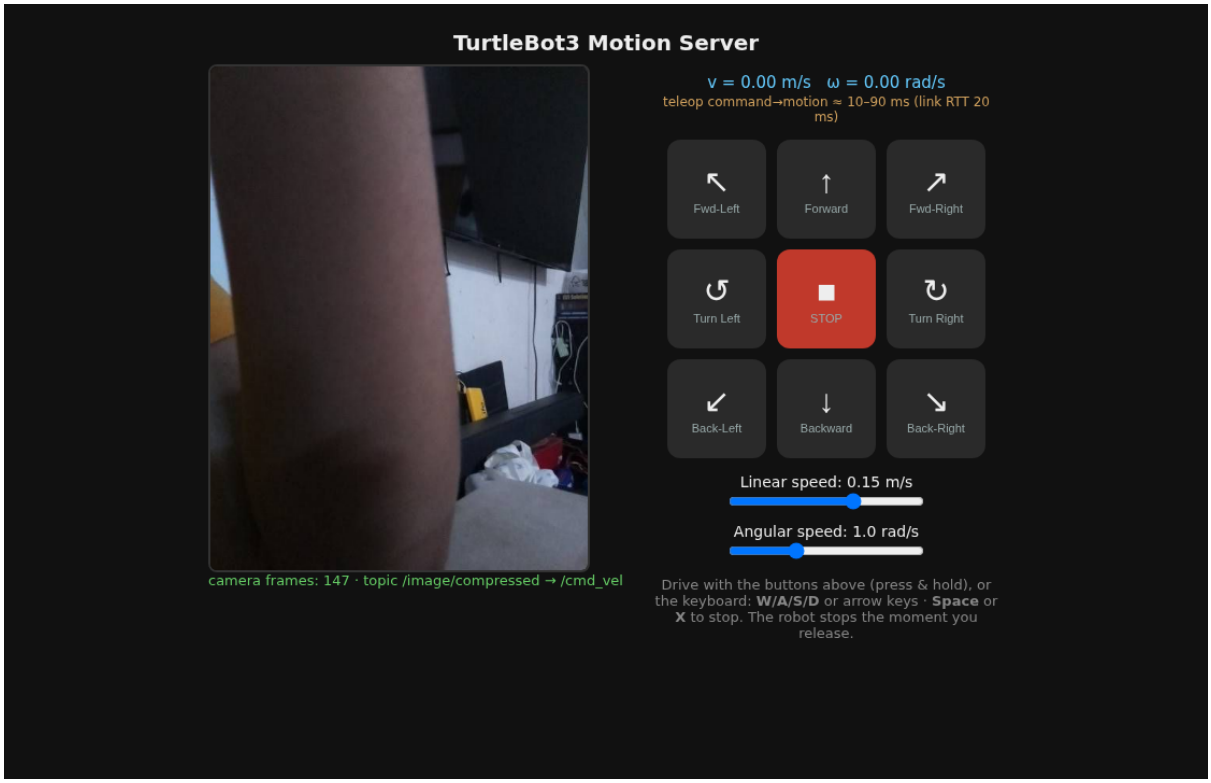

Figure 2: The web GUI. Camera feed on the left; full teleop control panel on the right with directional buttons, STOP, speed sliders, and the latency read-out.

6.5 Teleop latency read-out

The amber line under the velocity read-out estimates how long it takes for a button press to reach the robot as motion. It is built from a live measurement of the network **round-trip time** plus fixed allowances for internal processing — see Section 9 for the full explanation.

7 Problems Encountered and How They Were Solved

Getting the system working surfaced several real-world issues. They are documented here because they are common and instructive.

Nothing was visible at first. The robot was powered on but its ROS software had not been started. Powering on a robot is not the same as launching its programs — the motor and camera nodes must be started explicitly.

Domain ID mismatch. The robot's start-up file (`.bashrc`) set the ROS_DOMAIN_ID *five* times with conflicting values; the last one won, leaving the robot on domain 0 while the laptop was on 203. Because of the rule in Section 2.4, they could not see each other. The file was cleaned to set a single, consistent value (203).

Missing lidar model. Launching the robot failed with `KeyError: 'LDS_MODEL'` because the lidar type was not set in the environment. Setting `LDS_MODEL=LDS-01` fixed it.

SSH dropping on launch. Every time the robot software started, the remote terminal connection reset (a Wi-Fi hiccup under the sudden burst of network traffic). The fix was to launch the robot programs *detached*, so they keep running even if the terminal disconnects.

Multicast worry. The home Wi-Fi (a `192.168.68.x` network, typical of Google/Nest routers)

was suspected of blocking the "multicast" traffic ROS uses to discover nodes. A unicast fall-back configuration (`fastdds_unicast.xml`) was prepared, but testing showed multicast actually worked, so the default is used.

8 How to Run Everything

This is the complete operating procedure.

8.1 One-time setup

```
# Copy the configuration template and fill in your values
cp .env.example .env
nano .env # set ROBOT_IP, ROBOT_PASSWORD, ROS_DOMAIN_ID, etc.
```

8.2 Step 1 — Start the robot's software

This starts the motor driver and the camera on the robot, over SSH:

```
./scripts/robot_start.sh
```

It launches everything on the correct domain and waits for it to come up. You can confirm the robot's topics are visible from the laptop with:

```
export ROS_DOMAIN_ID=203
ros2 topic list # you should see /cmd_vel, /image/compressed, /scan, ...
```

8.3 Step 2 — Start the web server

```
./run_server.sh
```

8.4 Step 3 — Open the GUI and drive

Open `http://localhost:8000` in a browser (or `http://192.168.68.113:8000` from your phone on the same Wi-Fi). Drive with the buttons or keyboard. *Watch the robot while you drive.*

8.5 Step 4 — Shut down

```
# Stop the web server: press Ctrl-C in its terminal, or:
pkill -f motion_server.py

# Stop the robot's ROS nodes (robot stays powered on):
./scripts/robot_stop.sh
```

8.6 Step 5 — Power off the robot (optional)

Always shut the Raspberry Pi down cleanly before cutting power, to avoid corrupting its memory card:

```
ssh ubuntu@192.168.68.100 'sudo poweroff'
# wait ~20 s for the green light to stop blinking, then flip the power switch
```

9 Understanding Teleop Latency

Latency is the delay between doing something and seeing the effect. For teleop, it is the time from pressing a button to the wheels actually turning.

The most important and variable part is the **Round-Trip Time (RTT)**: how long a small network packet takes to travel from the laptop to the robot *and back*. The server measures this live (it appears as "link RTT" in the GUI). A typical value on this setup is about 20 ms; if the Wi-Fi degrades, this number climbs and driving feels sluggish.

A command only needs to travel *one way* (laptop → robot), which is about half the RTT. The GUI's estimate therefore adds up:

$$\text{latency} \approx \underbrace{\tfrac{1}{2}\,\text{RTT}}_{\text{network, one way}} + \underbrace{50\,\text{ms}}_{\text{command cycle (20 Hz)}} + \underbrace{\sim 30\,\text{ms}}_{\text{motor response}} .$$

For a 20 ms RTT this gives roughly 10–90 ms. Note this is a well-grounded *estimate*: only the RTT is measured directly; the other terms are known constants of the system.

10 Troubleshooting

Symptom	Likely cause and fix
`ros2 topic list` shows nothing from the robot	Robot software not started (run `robot_start.sh`), or domain mismatch (both must be `ROS_DOMAIN_ID=203`).
GUI says "waiting for camera" but the robot drives	Wrong camera topic; check `ros2 topic list` and `IMAGE_TOPIC` in `.env`.
Camera shows but robot will not move	Wrong command topic, or the robot's motor node is not running.
Robot launch fails: `KeyError: 'LDS_MODEL'`	Set `LDS_MODEL` (e.g. `LDS-01`) before launching.
SSH disconnects when starting the robot	Expected Wi-Fi hiccup; the scripts launch detached so the nodes survive. Just reconnect and check.
Driving feels laggy	Check the "link RTT" in the GUI; a high value means poor Wi-Fi.

Table 3: Common problems.

11 Repository Structure and Security

```
ttb-experiments/
  motion_server.py # the whole application (node + server + GUI)
  run_server.sh # start the server on the laptop (reads .env)
  scripts/
    robot_start.sh # start the robot's ROS nodes over SSH
    robot_stop.sh # stop the robot's ROS nodes
  fastdds_unicast.xml # optional fall-back for multicast-blocking networks
  .env.example # configuration template (safe to share)
  .env # YOUR real config incl. password (git-ignored!)
  robot/ # reference snapshot of the ROBOT-side files
    start_all.sh # launches bringup + camera on the robot
    turtlebot3_image_motion/
      turtlebot3_image_motion/image_publisher.py # the camera node
```

```
    srv/Motion.srv # a service definition (unused by this app)
    package.xml / CMakeLists.txt
  docs/
    turtlebot3_motion_server.tex / .pdf # this document
    images/gui.png
```

About `robot/`. Those files run on the robot, not the laptop, and are copied here only so this repository documents the whole system. They are a snapshot of commit `54fb8a2` of a separate project, `https://github.com/SuperMadee/AI361-MEX3`, which remains the source of truth — the robot builds from its own clone in `~/ros2_ws`, so editing the copies here changes nothing on the robot. Only the parts this project depends on were copied; the upstream package's Jetson-side AI stack (roughly 15,000 lines) is deliberately not duplicated.

Security note. The robot's SSH password and other real values live only in `.env`, which is listed in `.gitignore` and is therefore *never* committed or pushed to GitHub. Only `.env.example`, containing placeholders, is shared. Never put real passwords in a public repository — automated bots scan GitHub for leaked credentials within minutes.

A Environment Variable Reference

Variable	Description
`ROBOT_IP`	Robot's IP address on the Wi-Fi.
`LAPTOP_IP`	Laptop's IP address.
`ROBOT_USER`	SSH username on the robot.
`ROBOT_PASSWORD`	SSH password (secret; kept only in `.env`).
`ROS_DOMAIN_ID`	Shared discovery domain; must match on both machines.
`TURTLEBOT3_MODEL`	Robot variant (`burger`).
`LDS_MODEL`	Lidar model (`LDS-01`).
`IMAGE_TOPIC`	Topic the camera publishes on.
`IMAGE_TYPE`	`compressed`, `raw`, or `auto`.
`CMD_VEL_TOPIC`	Topic drive commands are published on.
`MAX_LIN`	Max linear speed (m/s).
`MAX_ANG`	Max angular speed (rad/s).
`SERVER_PORT`	Web server port.

B Command Cheat-Sheet

```
# --- laptop ---
./run_server.sh # start the web GUI server
pkill -f motion_server.py # stop it
export ROS_DOMAIN_ID=203; ros2 topic list # what the robot is publishing

# --- robot (over SSH) ---
./scripts/robot_start.sh # start motors + camera
./scripts/robot_stop.sh # stop them
ssh ubuntu@192.168.68.100 'sudo poweroff' # clean power-off
```